Quasigroups, Latin Squares, and the P2P OMI Citation Resolver

1. Purpose

This note explains how quasigroups and Latin squares can be used as an algorithmic model for OMI peer-to-peer citation discovery.

The goal is not to make OMI "about" abstract algebra. The algebraic names are handles for a deterministic computation pattern:

```
given two bounded coordinates,
resolve the third;
given a different pair,
recover the missing coordinate.
```

This is the useful part.

OMI needs exactly that pattern because its internal work is threefold:

```
1. Declare notation for citation.
2. Define citation by attestation and attribution.
3. Discover structure by interpolation and interpretation.
```

There is no fourth OMI phase. Application is downstream and outside OMI authority.

OMI constructs and annotates citations. It does not determine what an application must do with them.

---

2. Why Latin Squares Matter

A Latin square of order `n` is an `n × n` table filled with `n` symbols so that every symbol appears exactly once in each row and exactly once in each column.

That means every row-column pair resolves to exactly one symbol:

```
row × column → symbol
```

A finite quasigroup is the same structure read algebraically: its multiplication table, after removing row and column headers, is a Latin square. Conversely, a Latin square can be bordered with row and column labels to obtain a quasigroup table.

This is the part OMI can use.

The Latin-square property gives deterministic recovery:

```
row + column → symbol
row + symbol → column
column + symbol → row
```

So a Latin square is not just a table. It is a reversible finite resolver.

For OMI, that maps naturally to:

```
attestation + attribution → discovered citation point
attestation + discovered point → attribution
attribution + discovered point → attestation
```

That is why Latin squares may work P2P.

They give a way for peers to resolve missing citation coordinates without needing a central authority to interpret the whole graph.

---

3. OMI Three-Part Citation Model

OMI should be described as three internal phases.

Part 1 — Declaration

Declaration introduces the notation for citation.

Examples:

```
omi---imo
(NULL . NULL)
256-bit hexadecimal citation
fixed-width OMI frame
FΔ 00 1C 1D 1E 1F 20 FΔ
```

At this stage the system says:

```
this citation form exists
this address shape is declared
this notation can be carried
```

It does not yet decide application meaning.

Part 2 — Definition

Definition splits the citation into the two complementary halves:

```
128-bit attestation by declaration
128-bit attribution by definition
```

This gives the citation a bounded internal structure.

```
attestation:
  who/what declares the position

attribution:
  what the position is defined as
```

Together they form a 256-bit hexadecimal citation note.

The note is Dewey-like in the sense that it is a call number or locator, not merely a passive reference. It places a collaborative point into a navigable semantic geometry.

Part 3 — Discovery

Discovery uses interpolation and interpretation.

```
interpolation:
  discover positions between already defined points

interpretation:
  choose how the resolved citation is read in the current frame
```

At this phase a citation may be discovered as:

```
point
line
```

```
relation
hyperedge
surface projection
refinement cell
```

But OMI still does not decide the application.

The output is an annotatable citation surface, not an application command.

---

4. Why This Is Quasigroup-Like

A quasigroup is useful because it gives reversible local composition without requiring global associativity.

That matters because OMI does not need to be a group.

A group would require:

```
closed operation
identity
inverse
associativity
```

OMI does need closure and inverse-facing recovery, but it does not need one global associative operation over all possible semantic relations.

A quasigroup is weaker and more useful:

```
for every a and b, a * b resolves uniquely;
given any two of a, b, and a * b, the third can be recovered.
```

This matches citation discovery.

```
declaration * definition = discovered point
```

Given:

```
declaration + definition
```

the peer can discover the point.

Given:

```
declaration + point
```

the peer can recover the missing definition.

Given:

```
definition + point
```

the peer can recover the missing declaration.

That is the P2P value.

---

5. Latin Squares as P2P Citation Tables

In a P2P OMI network, each peer does not need to possess the entire semantic world.

Instead, peers can exchange bounded table fragments:

```
row fragment
column fragment
symbol fragment
orthogonal-array triple
transversal witness
intercalate witness
```

A Latin square can also be represented as an orthogonal array: a list of triples

```
(row, column, symbol)
```

with the property that any two columns contain all ordered pairs exactly once.

For OMI, those triples become:

```
(attestation, attribution, discovered-point)
```

So the P2P resolver can operate on triples:

```
(A, B, C)
```

where:

```
A = attestation coordinate
B = attribution coordinate
C = discovered citation point
```

The resolver law is:

```
any two determine the third
```

That is the computational heart of the Latin-square P2P idea.

---

6. Polybius Square as the Order-4 Governor

The Polybius gauge is the existing OMI version of this idea at nibble scale.

The nibble field is:

```
0 1 2 3
4 5 6 7
8 9 A B
C D E F
```

The row and column functions are:

```
row(x) = x / 4
col(x) = x mod 4
```

The control diagonals are:

```
D+ = {0,5,A,F}
D- = {3,6,9,C}
```

The active proof book records the finite Polybius facts:

```
XOR(D+) = 0
XOR(D-) = 0
SUM(D+) = 0x1E
SUM(D-) = 0x1E
SUM(D+) + SUM(D-) = 0x3C
SUM(0..F) = 0x78
15 * 16 = 240
```

These are proved on the Polybius gauge rail, not on the Delta16 replay rail.

So `0x3C` and `0x78` are not competing with `0x001D` and `0x1337`.

They belong to different rails:

```
0x3C / 0x78:
  Polybius gauge witnesses

0x001D / 0x1337:
  Delta16 runtime replay constants
```

The Polybius square gives the local 4-by-4 resolver.

The Latin square generalizes that pattern into a P2P resolver.

---

7. Metatron Scribe as Quasigroup Resolver

The Metatron Scribe can be modeled as quasigroup-like.

Its role is not to command the system. Its role is to preserve and interpolate structured citation relations.

```
Metatron Scribe:
  stores incidence triples
  records declaration/definition/discovery tables
  supports reversible lookup
  interpolates without changing authority
```

In the proof book, Metatron interpolation receives:

```
source address
destination address
already validated relation
```

and preserves the supplied decision. It cannot turn an unlawful decision into a lawful one.

That is already quasigroup-like behavior in spirit:

```
it moves through address relations
without changing the underlying decision class
```

In the P2P Latin-square model, Metatron becomes the scribe of triples:

```
A = attestation
B = attribution
C = discovered point
```

and preserves the resolver structure:

```
A * B = C
A \ C = B
C / B = A
```

The important property is not the symbol `*`.

The important property is deterministic recovery.

---

8. Steiner Quasigroup Rail

A Steiner quasigroup is especially relevant when the resolver is based on triple incidence.

A Steiner-style structure is useful when relations are naturally ternary:

```
point A
point B
third point C on their determined line
```

This resembles the Fano-plane part of OMI.

In OMI terms:

```
given two bounded citation points,
resolve the third incidence point
```

So a Steiner-like Metatron rail would be useful for:

```
Fano incidence
triple completion
bounded relation discovery
citation interpolation
P2P reconciliation
```

It gives an algorithmic rule:

```
two known endpoints determine the missing incidence witness
```

This is not yet saying every OMI relation is a Steiner quasigroup. It says that the Metatron Scribe may use a Steiner-quasigroup-like resolver for triple-incidence discovery.

That is a good fit for P2P because peers often have partial information.

One peer may have:

```
A and B
```

another may have:

```
A and C
```

another may have:

```
B and C
```

The resolver can reconcile the triple without requiring any one peer to own the whole graph.

---

9. Polyadic Quasigroup Rail

A binary quasigroup resolves:

```
a * b = c
```

A polyadic quasigroup generalizes this to more inputs:

```
f(a1, a2, ..., an) = z
```

with the important recovery property:

```
given all but one coordinate, recover the missing coordinate
```

This matches OMI's 256-bit citation frame better than a purely binary operation.

The citation is not just two small values. It is a fixed-width note:

```
16 × 16-bit pleths
```

split into:

```
8 × 16-bit declaration/attestation pleths
8 × 16-bit definition/attribution pleths
```

So the P2P resolver may need polyadic recovery:

```
given 15 fields, recover/check the 16th
given declaration half + partial definition, interpolate candidate
given definition half + partial attestation, recover missing declaration lane
```

A polyadic quasigroup is a good algorithmic model for this because it supports finite deterministic recovery across multiple slots.

In OMI vocabulary:

```
pleth:
  fixed finite field unit
```

```
nomogram:
  runtime alignment surface

polyadic quasigroup:
  deterministic recovery law across many pleths
```

---

10. Tetragrammatron Governor as Moufang-Loop-Like

The Polyharmonic Tetragrammatron Governor is not just a lookup table.

It governs reassociation.

That is why Moufang-loop-like behavior fits.

A Moufang loop is weaker than a group but stronger than an arbitrary loop. It permits certain lawful reassociations without requiring full associativity everywhere.

That matches OMI composition:

```
not globally associative
not arbitrary
not application-commanding
but lawfully reassociable inside the governor frame
```

In OMI terms:

```
Tetragrammatron Governor:
  preserves closure
  controls nonassociative composition
  allows lawful local reassociation
  keeps inverse-facing traversal coherent
```

This is important because OMI citations can compose in different groupings:

```
(A * B) * C
A * (B * C)
```

Those do not need to be globally equal.

But the governor can declare when a reassociation is lawful inside a bounded frame.

That is exactly the kind of job a Moufang-loop-like model can perform.

So:

```
Metatron:
  quasigroup / polyadic quasigroup / Steiner resolver

Tetragrammatron:
  Moufang-loop-like governor of lawful reassociation
```

---

11. Omicron Runtime Kernel

The Omicron is the runtime kernel surface.

Its base NULL Byte OMI-Ring is:

```
((0x00 . 0x20)
 (0x20 . 0x7F)
 (0x7F . 0xFF)
 (0xFF . 0x00))
```

The proof book gives the XOR witnesses:

```
00 XOR 20 = 20
20 XOR 7F = 5F
7F XOR FF = 80
FF XOR 00 = FF
```

and the closure:

```
20 XOR 5F XOR 80 XOR FF = 00
```

It also proves the general finite closed-path law:

```
(A0 XOR A1)
XOR (A1 XOR A2)
XOR ...
```

```
XOR (An XOR A0)
= 0
```

for finite closed paths.

This gives the Omicron base kernel:

```
closed byte-ring
null/readable/seal/saturation cycle
impotent projection surface
```

The Gnomic Gauge OMI-Ring extends that with an identity-bearing gauge:

```
FΔ 00 1C 1D 1E 1F 20 FΔ
```

This supports innocuous projection:

```
identity-bearing readable surface
harmless structured exposure
no application force
```

So:

```
Base Omicron NULL Byte Ring:
  Impotent Surface Projection

Gnomic Gauge OMI-Ring:
  Innocuous Surface Projection
```

---

12. P2P Latin-Square Resolver Design

A practical P2P resolver can be designed around Latin-square triples.

Local triple

```
(A, B, C)
```

where:

```
A = attestation coordinate
B = attribution coordinate
C = discovered citation point
```

Resolver operations

```
resolve(A, B) -> C
recover_definition(A, C) -> B
recover_attestation(B, C) -> A
```

Peer storage

A peer may store any of:

```
row shard:
  all triples for one attestation coordinate

column shard:
  all triples for one attribution coordinate

symbol shard:
  all triples resolving to one discovered point

transversal shard:
  one representative from each row and column

intercalate shard:
  small 2×2 consistency witness

orthogonal-array shard:
  normalized list of (A, B, C) triples
```

P2P exchange

Peers exchange bounded shards, not entire worlds.

```
peer sends row fragment
peer sends column fragment
peer sends symbol witness
peer sends missing coordinate request
peer verifies Latin-square uniqueness locally
```

Deterministic merge

A merge is accepted only when it preserves:

```
one symbol per row
one symbol per column
unique recovery from any two coordinates
bounded frame size
declared gauge closure
```

If a merge would duplicate a symbol in a row or column, it is not a valid table merge for that frame.

That does not mean the external idea is false. It means that particular fragment does not fit that bounded resolver frame.

---

13. Latin-Square P2P and OMI Phases

The Latin-square resolver maps to the three OMI phases:

```
Declaration:
  publish row/column/symbol coordinate scheme

Definition:
  bind attestation and attribution halves

Discovery:
  recover missing coordinates, interpolate triples, interpret surface role
```

No application decision occurs.

The P2P network can discover:

```
this citation point exists
this point is recoverable from these two coordinates
this interpolation is consistent
this projection is harmless or inert
```

But it does not decide:

```
this app must render it
this OS must execute it
```

```
this user must accept it
this world-state must change
```

That boundary is crucial.

---

14. Innocuous and Impotent Surface Projections

The algebraic resolver produces surface projections, but OMI must keep those projections bounded.

Impotent Surface Projection

Produced by the base Omicron NULL Byte Ring.

```
readable / inspectable
closed
inert
cannot instantiate application effects
```

It is useful for:

```
debugging
annotation
null closure display
address inspection
safe documentation
```

Innocuous Surface Projection

Produced by the Gnomic Gauge OMI-Ring.

```
identity-bearing
structured
readable
harmless
still not application-authoritative
```

It is useful for:

```
collaboration frames
semantic surfaces
```

```
P2P discovery views
non-commanding graph/lattice projection
```

The difference:

```
Impotent:
  cannot act

Innocuous:
  may inform, but still must not command
```

---

15. Why This Helps P2P

Latin-square/quasigroup structure is useful for P2P OMI because it avoids central lookup.

A central registry says:

```
ask the server what this citation means
```

A Latin-square resolver says:

```
given bounded coordinates,
compute the missing coordinate
```

That lets peers work with partial state:

```
Peer 1 knows A and B.
Peer 2 knows A and C.
Peer 3 knows B and C.

All three can reconcile the same triple.
```

This gives OMI a deterministic P2P discovery surface:

```
bounded
finite
reversible
shardable
```

```
merge-checkable
projection-safe
```

That is exactly the kind of structure needed for a pseudo-persistent semantic hypergraph or topological lattice.

---

16. Proposed OMI Role Table

OMI Role	Algebraic Analogy	Computational Meaning
Omicron	Runtime kernel / closure loop	Frames null/readable/seal/saturation closure
NULL Byte OMI-Ring	Closed path	Produces Impotent Surface Projection
Gnomic Gauge OMI-Ring	Loop-like identity frame	Produces Innocuous Surface Projection
Polybius Square	Order-4 finite resolver	Nibble-scale row/column/diagonal gauge
Metatron Scribe	Quasigroup / Steiner / polyadic quasigroup	Records and interpolates reversible incidence triples
Tetragrammatron Governor	Moufang-loop-like governor	Controls lawful reassociation and closure
OMI Citation	256-bit call number	Bounded declaration/definition note
OMI P2P Discovery	Latin-square shard reconciliation	Peers compute missing citation coordinates

---

17. Minimal Algorithm

```
Input:
  citation fragment(s)

Step 1 — Declare:
  identify the notation frame and field widths

Step 2 — Define:
  split into attestation and attribution coordinates

Step 3 — Discover:
  use Latin-square/quasigroup resolver to compute missing coordinate
```

```
Step 4 — Bound:
  verify row uniqueness, column uniqueness, symbol uniqueness, and closure

Step 5 — Project:
  emit an Impotent or Innocuous Surface Projection

Stop:
  do not decide application behavior
```

In pseudocode:

```
declare(frame)

A = attestation(frame)
B = attribution(frame)

C = latin_resolve(A, B)

if latin_unique(A, B, C) and closure_ok(frame):
    emit_surface_projection(C)
else:
    reject_fragment_for_this_frame
```

For recovery:

```
recover_definition(A, C) -> B
recover_attestation(B, C) -> A
```

For P2P merge:

```
merge(shard1, shard2):
    combine triples
    reject if any row repeats a symbol
    reject if any column repeats a symbol
    reject if any pair resolves ambiguously
    otherwise accept bounded shard
```

---

18. Canon Statement

OMI can use Latin squares P2P because a Latin square is a finite deterministic resolver: every row-column pair determines one symbol, and any two coordinates can recover the third.

In OMI terms, attestation and attribution resolve a discovered citation point. Any two of attestation, attribution, and discovered point can recover the missing coordinate.

The Polybius Square is the nibble-scale governor already present in the Tetragrammatron rail. Latin squares generalize that resolver pattern to P2P citation discovery.

Metatron acts as the quasigroup-like scribe of incidence triples. Tetragrammatron acts as the Moufang-loop-like governor of lawful reassociation and closure. Omicron supplies the runtime kernel and closed byte-ring surface.

The result is a deterministic, finite, shardable P2P method for constructing and discovering citation points in a semantic hypergraph or topological lattice.

OMI does not decide application behavior. It declares, defines, discovers, and projects only innocuous or impotent surfaces.